

PromptVault Vulnerability Remediation

Student Name: Yasir Celtik

Student Number: 25277644

Email: yasir.celtik@ucdconnect.ie

Application Assessed: PromptVault

Assessment Author: Katara Winter (25275037)

Table of Contents

1. Executive summary	3
2. Implementation Roadmap by Severity	3
3. Peer-Report Findings	4
3.1. P-01 — Plaintext password storage and raw string comparison	4
3.2. P-02 — Known vulnerable third-party dependencies	5
3.3. P-03 — Prompt edit IDOR	6
3.4. P-04 — Messages can be posted to another user's conversation	7
3.5. P-05 — Conversation can be started from another user's private prompt	9
3.6. P-06 — Disabled users retain authenticated sessions	11
3.7. P-07 — Weak password policy	12
3.8. P-08 — Missing CSRF protection on state-changing requests	13
3.9. P-09 — Unrestricted authentication attempts	14
3.10. P-10 — Client-side-only input validation	16
3.11. P-11 — Personally identifiable information in logs	17
3.12. P-12 — Log injection through untrusted values	17
3.13. P-13 — Missing Content-Security-Policy	18
3.14. P-14 — Missing anti-clickjacking protection	19
3.15. P-15 — Session cookie lacks explicit SameSite policy	19
3.16. P-16 — Verbose exception and framework error disclosure	20
3.17. P-17 — Timing-sensitive plaintext password comparison	21
4. Lecturer guideline additions	21
4.1. T-01 — DB and administrator credentials hard-coded inside application.config ...	21
5. OWASP Top 10:2025 Coverage Matrix	23
6. Existing protections and false positives	25
7. Conclusion	25

1. Executive summary

The Promptvault application currently contains all 17 identified findings from the peer vulnerability report. The OWASP 10 list provided from the lecturer is planned to be implemented after the peer reviews comments as there is an overlap between both. The lecturers specification also exposes additional gaps in session handling, application configuration, http transport security, auditing and secure design.

Remediation began with ordering the items based on their criticality which was also done by the peer review as an order of importance.

Promptvault now contains the improvements from all 17 identified vulnerabilities.

This document contains an extracted format that allowed me to work through step by step to remediate the problems

Each section contains a table on the details of what OWASP and CWE the vulnerability maps to, the location in code provided from the peer assessment, a status for work tracking and a Severity from High, Medium to Low. Underneath the current evidence, risk posed and planned mitigation has been paraphrased from the peer assessment due to brevity.

Each section ends on an Implementation guide provided by the student and with the help of ChatGPT.

The Verification and report evidence section of each vulnerability contains information on what was done to mitigate the vulnerability.

2. Implementation Roadmap by Severity

1. **Update dependencies:** gradually upgrade the version of Spring up to the point where we have as few High severity dependencies as possible. Verify with Snyk
2. **Implement Spring Security:** Implement Bcrypt for password hashing in the database. Securely migrate existing users to encrypted passwords.
3. **CSRF mitigation:** Enable built in CSRF protection in Spring
4. **HTTP Header settings:** Further mitigate clickjacking, spoofing, and authentication by tightening the scope of HTTP request headers
5. **Enforce ownership checks at the repository level:** Implement checks for record ownership on CRUD operations to limit exposure of data not owned by users.
6. **Invalidate disabled accounts:** Implement Bucket4j for throttling account guessing. Add in a retry count and block multiple incorrect guesses for a period of time.
7. **Validate user inputs on the server side:** Implement spring data validation and annotations to enforce safe inputs from the user.

8. **Harden logs and configuration.** Remove PII and special control characters from log statements, externalize secrets into environments, and split development from production application.properties.
9. **Fail safely.** Add generic exception handling, create unit tests to confirm transactional logic.

3. Peer-Report Findings

The identifiers P-01 through P-17 follow the peer report's severity groups and order. I've created these finding sections as a to-do list and implementation guide from the assessors document. They take precedence over lecturer-only additions as there is overlap between some of them.

3.1. P-01 — Plaintext password storage and raw string comparison

Severity	Critical
Status	Complete
Source	Peer report, Critical finding 1
Location	AuthService.java:21–46, DataSeeder.java, User.password
CWE	CWE-256
OWASP 2025	Reported A06; assignment mapping A04 Cryptographic Failures and A07 Authentication Failures

Current evidence. Registration writes `request.password()` directly and login compares it with `String.equals`. Seed accounts also contain reusable plaintext passwords.

Risk. A database, heap, backup, or session disclosure immediately reveals reusable credentials.

Planned mitigation and rationale. Use Spring Security's BCrypt password encoder and retain only password hashes. BCrypt is salted and deliberately expensive, reducing the value of a database dump and also removing direct plaintext comparison.

Implementation guide. Add Spring Security and a PasswordEncoder; store a minimal principal; encode registration and seed passwords. Run a one-time transactional migration that detects non-BCrypt database values, hashes them, and then remove the migration. Do not retain a plaintext or {noop} fallback.

Verification and report evidence.

- Confirmed that new registrants have their password hashed in the database Figure 1
- Confirmed that the CommandLineRunner now checks the database for unhashed passwords and hashes them

- Added unit tests to confirm that hashing takes place in the registration process and unhashed values

are compared in the login process.

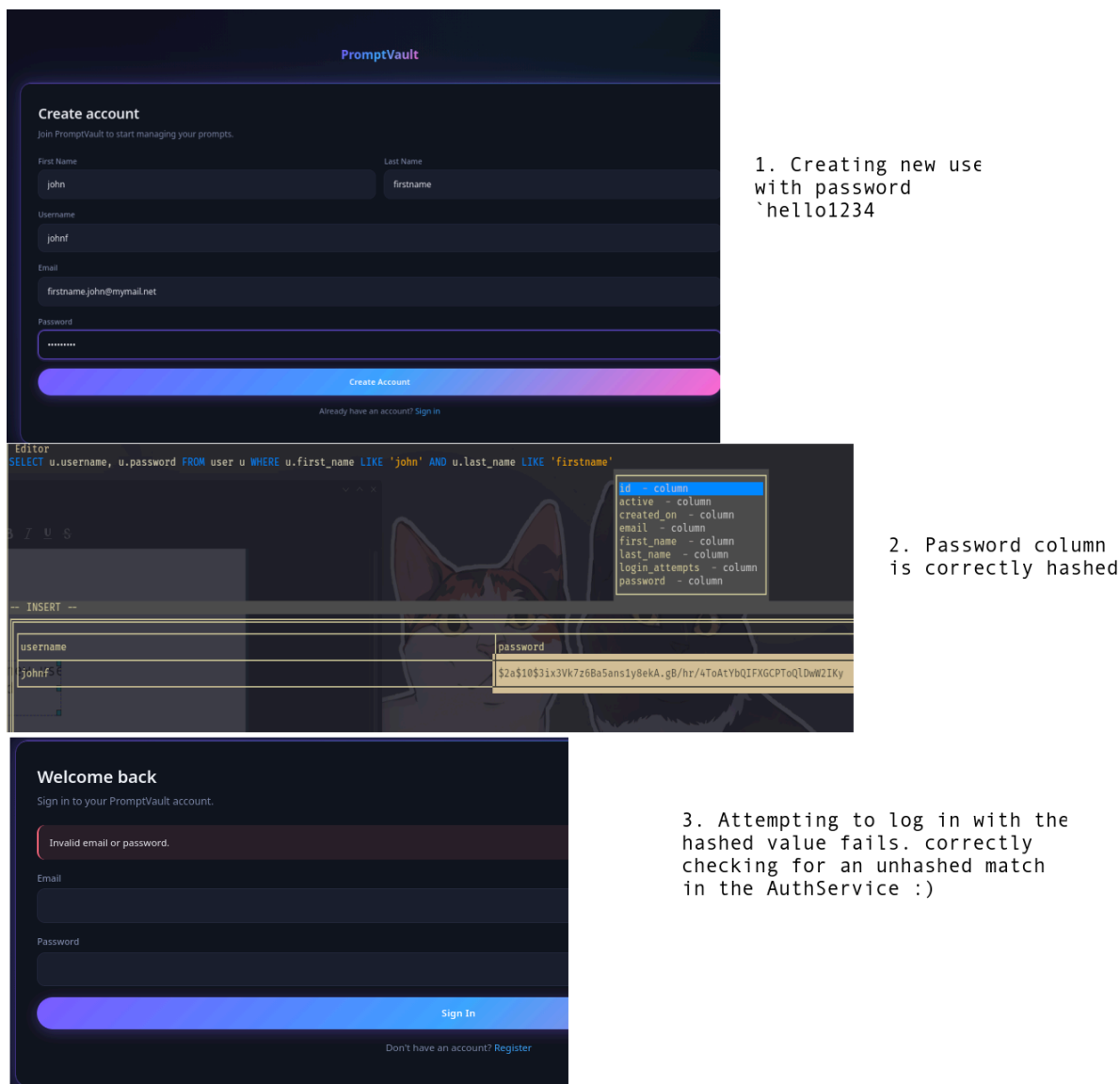


Figure 1: Image showing creating a new user with the password 'hello1234' and attempting to log in with the hashed value to show that the login system is now using bcrypt hashing to verify password as opposed to plain string matches.

3.2. P-02 — Known vulnerable third-party dependencies

Severity	Critical
Status	Complete
Source	Peer report, Critical finding 2
Location	pom.xml Spring Boot parent 3.5.14 and resolved transitive dependencies

CWE	CWE-1395 / CWE-1104
OWASP 2025	A03 Software Supply Chain Failures

Current evidence. The peer’s Snyk and OWASP Dependency-Check results report critical vulnerable components. The parent version remains 3.5.14.

Risk. A published dependency vulnerability may make application compromise possible even when application code is correct.

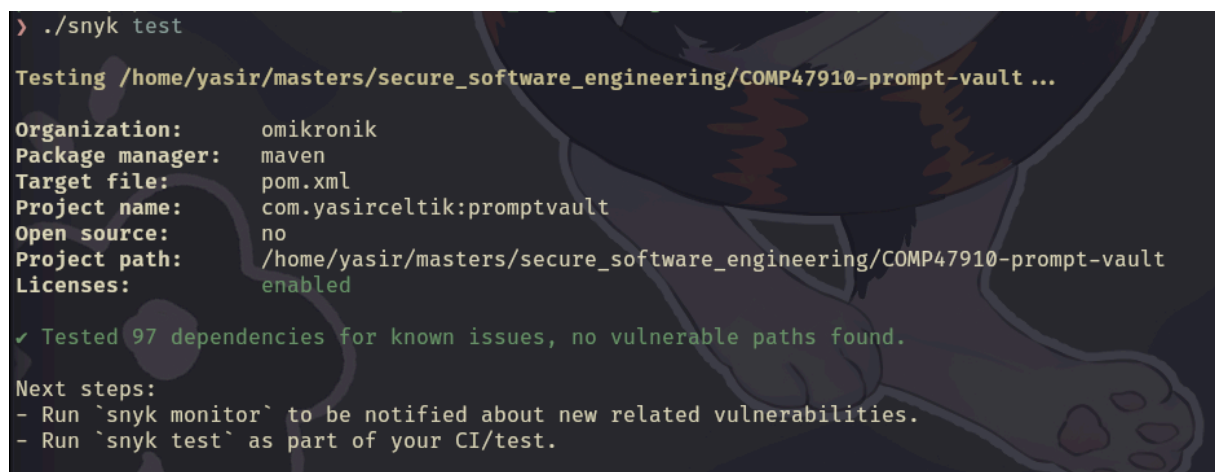
Planned mitigation and rationale. Upgrade to Spring Boot 3.5.16 first because it is the least disruptive patched 3.5.x baseline, remove unnecessary runtime dependencies, and treat a supported 4.x migration as follow-up work. Dependency scans make vulnerable versions visible before release.

Implementation guide. Update the parent only after recording the current tree; review release notes; remove production devtools

Verification and report evidence. To remediate dependency vulnerabilities, a path following Snyk’s guidance for minimum required versions was traced. This went from:

1. 3.5.14 -> 3.5.16
2. 3.5.16 -> 4.0.0
3. 4.0.0 -> 4.0.8

This required no code changes as modern practices were followed such as using jakarta as a dependency as opposed to javax which meant that no code upgrades were required.



```

> ./snyk test

Testing /home/yasir/masters/secure_software_engineering/COMP47910-prompt-vault ...

Organization:    omikronik
Package manager: maven
Target file:     pom.xml
Project name:    com.yasirceltik:promptvault
Open source:     no
Project path:    /home/yasir/masters/secure_software_engineering/COMP47910-prompt-vault
Licenses:        enabled

✓ Tested 97 dependencies for known issues, no vulnerable paths found.

Next steps:
- Run `snyk monitor` to be notified about new related vulnerabilities.
- Run `snyk test` as part of your CI/test.

```

Figure 2: Snyk output of dependency vulnerabilities being fixed on Spring version 4.0.8

3.3. P-03 — Prompt edit IDOR

Severity	High
Status	Complete
Source	Peer report, High finding 1

Location	PromptController edit GET/POST; PromptService.editPrompt
CWE	CWE-639 / CWE-284
OWASP 2025	A01 Broken Access Control

Current evidence. Both edit endpoints fetch by attacker-controlled ID after checking only that a session exists. The service accepts a user but never verifies ownership.

Risk. Any authenticated user can read and overwrite another user's private prompt.

Planned mitigation and rationale. Load editable prompts using both prompt ID and authenticated owner ID, and reject access in the service as well as at the route. This prevents ID manipulation from selecting another user's object.

Implementation guide. Prompt editing safeguard was added such that when an edit page for a prompt is clicked, the repository now retrieves prompts by id and owner, as opposed to from just the prompt id as such:

```
Optional<Prompt> findByIdAndOwner(Long id, User owner);
```

Listing 1: Repository definition to retrieve prompt by prompt id and by owner

For administrators, who are should retain the ability to delete any prompt, a check for the role of the session making the request was added.

Verification and report evidence. Unit tests were created for PromptService. Alongside manual testing of the functionality.

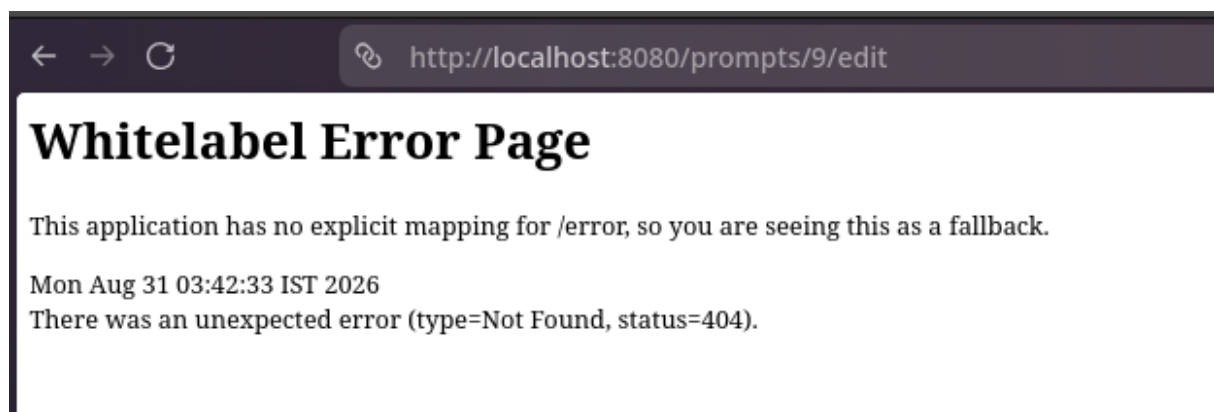


Figure 3: Logged in to user John, manually editing the url to an id they do not own, redirects correctly to a 404.

3.4. P-04 — Messages can be posted to another user's conversation

Severity	High
Status	Complete
Source	Peer report, High finding 2

Location	ConversationController.sendMessage; ConversationService.sendMessage
CWE	CWE-639 / CWE-284
OWASP 2025	A01 Broken Access Control

Current evidence. sendMessage fetches the conversation by ID and never compares its owner with the authenticated user.

Risk. An attacker can alter another user's history, trigger flags, and inject content into their conversation.

Planned mitigation and rationale. Resolve the conversation through an ID-and-owner query inside the transactional service before creating either message.

Implementation guide. Pass the authenticated principal ID to the service; retrieve with findByIdAndOwnerId; ensure no user or simulated-agent message is written on denial.

Implementation detail very similar to P-03. Relevant repository now pulls from an owner aware request.

```
Optional<Conversation> findByIdAndOwnerId(Long id, Long ownerId);
```

Listing 2: Repository definition to retrieve conversation by conversation id and by owner

Verification and report evidence.

```

> curl -i \
-b "JSESSIONID=68E1ACF3CF4FB92F2AD3971DE70F5013" \
http://localhost:8080/conversations/7

HTTP/1.1 500
X-Content-Type-Options: nosniff
X-XSS-Protection: 0
Cache-Control: no-cache, no-store, max-age=0, must-revalidate
Pragma: no-cache
Expires: 0
X-Frame-Options: DENY
Content-Type: application/json
Transfer-Encoding: chunked
Date: Mon, 31 Aug 2026 03:18:57 GMT
Connection: close

{"timestamp":"2026-08-31T03:18:57.572Z","status":500,"error":"Internal
Server Error","path":"/conversations/7"}

```

Listing 3: Curl request that would previously pass. This is a session ID from a test user that does not own conversation with ID 7.

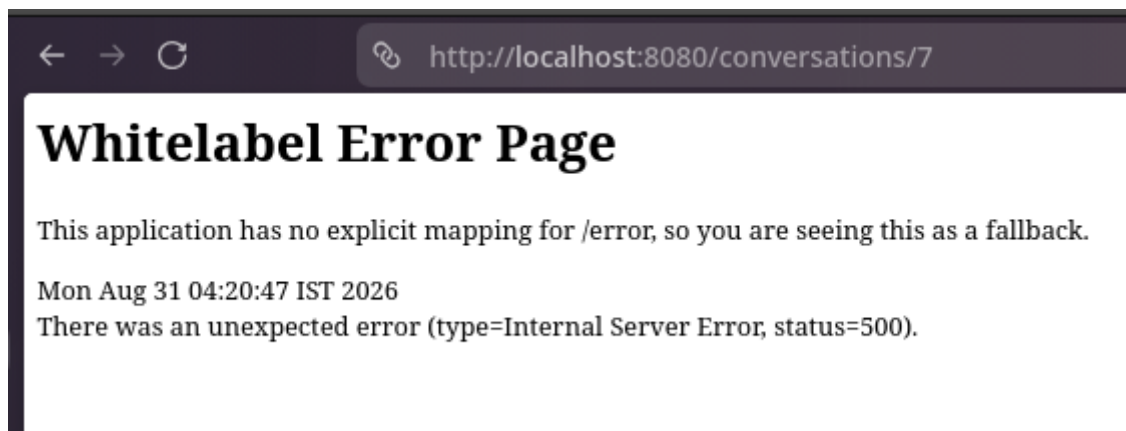


Figure 4: Likewise, manual link editing for conversation id 7 also returns the same 500 error due to not being allowed to view.

3.5. P-05 — Conversation can be started from another user's private prompt

Severity	High
Status	Complete
Source	Peer report, High finding 3
Location	ConversationController.startConversation; ConversationService.createConversation
CWE	CWE-639 / CWE-284

Current evidence. Any existing prompt ID is accepted; neither ownership nor SHARED visibility is checked.

Risk. An attacker can associate a conversation with a private prompt and disclose its title or future prompt-derived behavior.

Planned mitigation and rationale. Permit prompt use only when the current user owns it or its persisted visibility is SHARED. Never trust the hidden promptId field or UI filtering.

Implementation guide. Similar to P-03, P-04, in this case a hand rolled query was needed to get the prompt with some id IF it is owned by the current user OR visibility is shared.

Verification and report evidence.

```
> curl -i \
-b "JSESSIONID=87AB1F44237512F1A00932B3ECFB64D0" \
-X POST \
-d "promptId=9" \
http://localhost:8080/conversations/start

HTTP/1.1 404
X-Content-Type-Options: nosniff
X-XSS-Protection: 0
Cache-Control: no-cache, no-store, max-age=0, must-revalidate
Pragma: no-cache
Expires: 0
X-Frame-Options: DENY
Content-Type: application/json
Transfer-Encoding: chunked
Date: Mon, 31 Aug 2026 03:56:31 GMT

{"timestamp":"2026-08-31T03:56:31.526Z","status":404,"error":"Not Found","path":"/conversations/start"}↵
```

Listing 4: This Curl request was used as a template to test creation of prompts.

3 cases were tested

- user A to user B
- user A to admin
- admin to user A

In each case, the promptId was changed to one owned by the target user. all 3 curl cases passed.

3.6. P-06 — Disabled users retain authenticated sessions

Severity	High
Status	Complete
Source	Peer report, High finding 4
Location	SessionService; session user object; UserService.setActive
CWE	CWE-613
OWASP 2025	A07 Authentication Failures; related A01

Current evidence. The whole User entity is cached in the HTTP session and account status is checked only during login.

Risk. An administrator cannot promptly revoke a compromised or disabled account.

Planned mitigation and rationale. Use a minimal Spring Security principal and enforce current active status. Expire that user's sessions on disablement and retain a per- request active check as defence in depth.

Implementation guide. Register authenticated sessions, associate them with a stable user identifier, expire matching sessions in the admin disable transaction, and clear the security context if the database account is no longer active.

Verification and report evidence. Disabled a user from a second session and confirm the user's next request is rejected while unrelated sessions remain active.

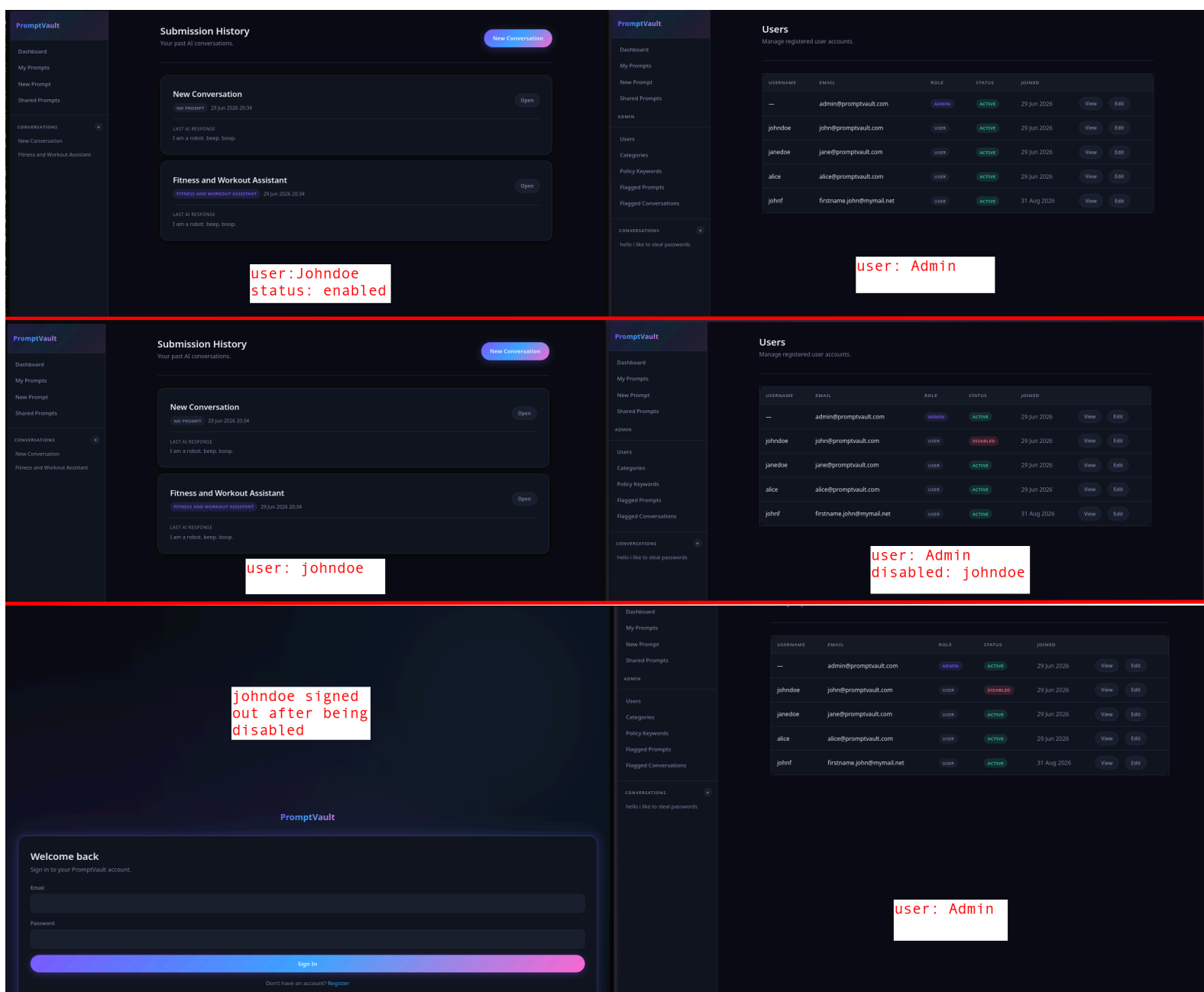


Figure 5: Flow of Admin (Right) disabling user Johndoe (Left) who is sent to the login screen on their next action after being disabled.

3.7. P-07 — Weak password policy

Severity	High
Status	Complete
Source	Peer report, High finding 5
Location	RegisterRequestDto; AuthController.register; registration template
CWE	CWE-521
OWASP 2025	A07 Authentication Failures

Current evidence. The server accepts any password, including an empty or one- character value. Browser required is the only constraint.

Risk. Weak credentials are quickly guessed through dictionary or credential attacks.

Planned mitigation and rationale. Enforce a server-side minimum length and maximum supported length, allow passphrases, and reject known-compromised choices if feasible. Avoid composition rules that encourage predictable substitutions.

Implementation guide. Add Bean Validation to the registration DTO and controller; use a 12-character minimum and a bounded maximum compatible with BCrypt; return field-level errors without echoing the password. Apply equivalent policy to seeded accounts.

Verification and report evidence. Added a number of unit tests to test all cases from the implementation step. Added form validation to the html registration

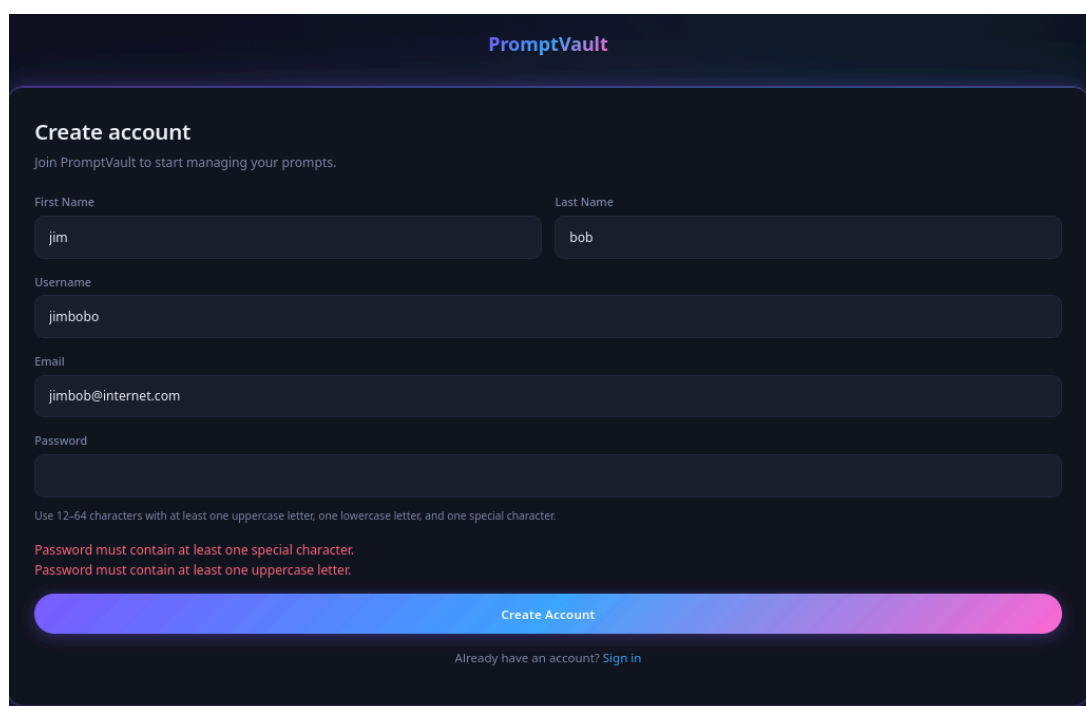
The image shows a web registration form for 'PromptVault'. The form is titled 'Create account' and includes a sub-header 'Join PromptVault to start managing your prompts.' It contains several input fields: 'First Name' (with 'jim' entered), 'Last Name' (with 'bob' entered), 'Username' (with 'jimbobo' entered), 'Email' (with 'jimbob@internet.com' entered), and 'Password'. Below the password field, there are two red error messages: 'Password must contain at least one special character.' and 'Password must contain at least one uppercase letter.' At the bottom of the form is a large, rounded button labeled 'Create Account' and a link that says 'Already have an account? Sign in'.

Figure 6: Registration form updated with form validation rules.

3.8. P-08 — Missing CSRF protection on state-changing requests

Severity	High
Status	Complete
Source	Peer report, High finding 6
Location	All POST forms and endpoints
CWE	CWE-352
OWASP 2025	Reported A07; assignment mapping A08 Software or Data Integrity Failures

Current evidence. Spring Security is absent and POST actions accept only the session cookie; forms have no CSRF token.

Risk. A malicious site can cause a logged-in browser to create, edit, delete , submit, or disable resources.

Planned mitigation and rationale. Enable Spring Security’s session-backed CSRF protection and require a valid token for every unsafe method. SameSite cookies provide defence in depth but do not replace CSRF validation.

Implementation guide. Keep CSRF enabled; render tokens in all Thymeleaf POST forms ; provide a safe access-denied response; do not exempt admin or delete endpoints. Keep all state changes off GET routes.

Verification and report evidence. All forms modified to explicitly include csrf token. a suite of unit tests generated by Chatgpt is used to verify CSRF functionality

3.9. P-09 — Unrestricted authentication attempts

Severity	Medium
Status	Complete
Source	Peer report, Medium finding 1; its description incorrectly repeats the CSRF heading
Location	AuthController.login; unused User.loginAttempts
CWE	CWE-307
OWASP 2025	A06 Insecure Design and A07 Authentication Failures

Current evidence. The login endpoint processes unlimited attempts and loginAttempts is never updated.

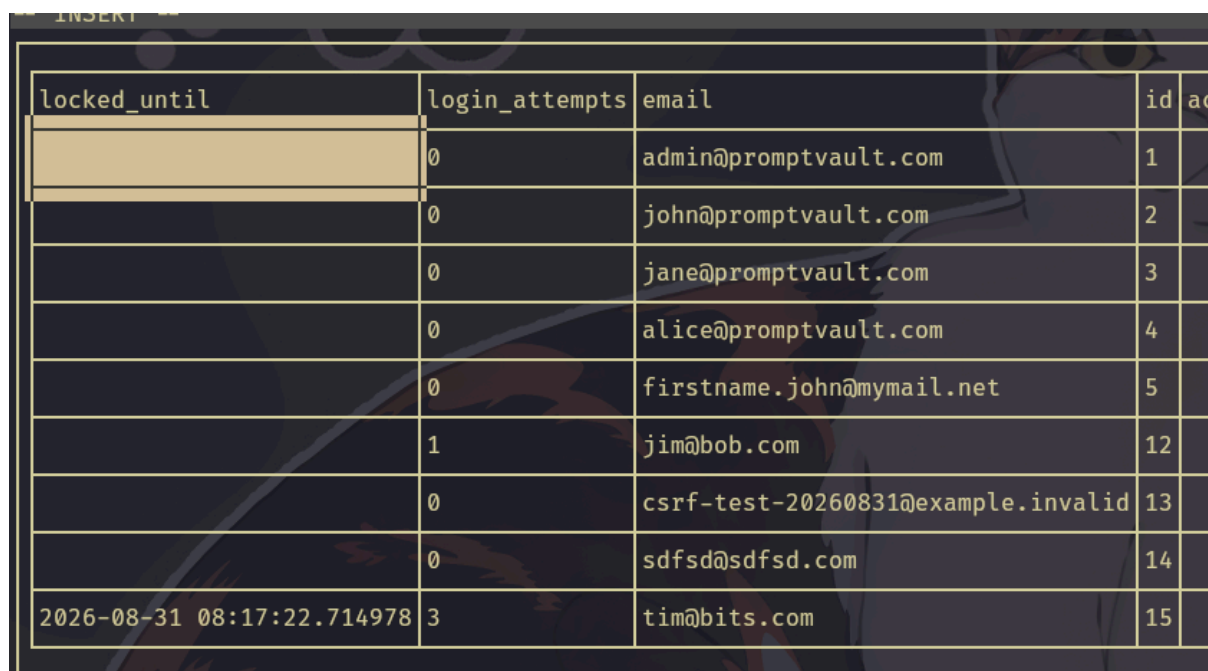
Risk. Attackers can perform rapid password guessing and credential stuffing.

Planned mitigation and rationale. Combine per-client throttling with temporary account delay/ lockout and generic responses with Bucket4j. Temporary limits reduce guessing speed without creating a permanent username-based denial of service.

Implementation guide. Choose and document limits, for example five failed account attempts followed by a 15-minute lock, plus a bounded IP-based request bucket . Reset counters after successful authentication and return 429 with Retry-After for transport throttling.

Verification and report evidence.

Implemented a 15 minute lockout system. This works on a login attempts system this gets reset after the timer is cleared by waiting or an admin. Each user is given 3 attempts at logging in.



The image shows a screenshot of a database table with columns: locked_until, login_attempts, email, id, and ac. The table contains 15 rows of user data. The user 'tim@bits.com' (id 15) is highlighted in yellow, showing a locked_until timestamp of '2026-08-31 08:17:22.714978' and 3 login attempts. Other users include 'admin@promptvault.com', 'john@promptvault.com', 'jane@promptvault.com', 'alice@promptvault.com', 'firstname.john@mymail.net', 'jim@bob.com', 'csrf-test-20260831@example.invalid', and 'sdfsds@sdfsds.com'.

locked_until	login_attempts	email	id	ac
	0	admin@promptvault.com	1	
	0	john@promptvault.com	2	
	0	jane@promptvault.com	3	
	0	alice@promptvault.com	4	
	0	firstname.john@mymail.net	5	
	1	jim@bob.com	12	
	0	csrf-test-20260831@example.invalid	13	
	0	sdfsds@sdfsds.com	14	
2026-08-31 08:17:22.714978	3	tim@bits.com	15	

Figure 7: Showing user tim@bits.com, with a 15 minute lockout timer.

As per suggestion of of assessor. Bucket4j was implemented to throttle request limits. For testing purposes, 10 requests per minute was set to make it simple to hit.

```

@Component
public class LoginRateLimitFilter extends OncePerRequestFilter {

    private static final int REQUEST_LIMIT = 10;

    private static final Duration REFILL_PERIOD =
        Duration.ofMinutes(1);

    private final Cache<String, Bucket> buckets =
        Caffeine.newBuilder()
            .expireAfterAccess(
                30,
                TimeUnit.MINUTES
            )
            .maximumSize(10_000)
            .build();

    ...

```

Listing 5: Partial sample of Bucket4j rate limiting code.

3.10. P-10 — Client-side-only input validation

Severity	Medium
Status	Complete
Source	Peer report, Medium finding 2
Location	All request DTOs/controllers; prompt, message, category, keyword, registration fields
CWE	CWE-602 / CWE-20
OWASP 2025	A05 Injection and A06 Insecure Design

Current evidence. DTOs have no validation annotations. Empty policy keywords cause every string to match, and oversized values can reach database constraints.

Risk. Malformed input can corrupt policy behavior, trigger exceptions, or store unusable records.

Planned mitigation and rationale. Treat browser constraints as usability only and enforce format, required fields, allow-listed enums, and database-aligned lengths on the server.

Implementation guide. Add spring-boot-starter-validation; constrain each DTO; use @Valid and BindingResult; normalize email and keyword values; reject blank keywords and invalid category references; return field errors without persisting partial data.

Verification and report evidence. Submit blank, whitespace, oversized, duplicate, invalid enum, missing relation, CR/LF, and direct curl payloads for every write endpoint.

Implemented validation on DTO objects as per assessment recommendations. Eliminating critical bugs such as blank Policy Keywords that would flag every single prompt

```
public record CreatePolicyKeywordDto(  
    @NotBlank(message = "Keyword cannot be blank.")  
    @Size(max = 255, message = "Keyword must not exceed 255 characters.")  
    @Pattern(regexp = "^[^\\r\\n]*$", message = "Keyword must not contain  
line breaks.")  
    String content) {  
}
```

Listing 6: CreatePolicyKeywordDto with validation annotations to combat invalid inputs.

3.11. P-11 — Personally identifiable information in logs

Severity	Medium
Status	Complete
Source	Peer report, Medium finding 3
Location	AuthController email logs; UserService.setActive
CWE	CWE-532
OWASP 2025	A09 Security Logging and Alerting Failures

Current evidence. Login attempts and account-status changes write complete email addresses to console and rolling files.

Risk. Log access exposes a user directory and increases privacy impact.

Planned mitigation and rationale. Log stable internal IDs or non-reversible correlation values and retain only the minimum information needed for investigation.

Implementation guide. Remove raw email, prompt text, secrets, session IDs, and credentials from events; define retention and file permissions; review exception logging for request data.

Verification and report evidence. Manual inspection removed all PII's from log messages

3.12. P-12 — Log injection through untrusted values

Severity	Medium
Status	Complete
Source	Peer report, Medium finding 4
Location	Authentication, category, keyword, prompt, and conversation log calls

CWE	CWE-117
OWASP 2025	A05 Injection and A09 Security Logging and Alerting Failures

Current evidence. User-controlled values are interpolated without neutralizing CR, LF, tab, or control characters.

Risk. An attacker can forge events, split log lines, or conceal malicious activity.

Planned mitigation and rationale. Prefer identifiers over user text; when text is necessary, centralize CR/LF/control-character neutralization and length bounding. Structured logs further separate data from event syntax.

Implementation guide. Create one tested sanitization or structured-audit path; do not rely on scattered replacements; cap values and preserve an explicit truncation marker.

Verification and report evidence. Send CR/LF/tab/control characters and long values, then assert one physical event is emitted and no attacker-created timestamp or level is interpreted.

Identified points where user inputs are being logged

- PromptService: prompt title
- ConversationService: conversation title
- PolicyKeywordService: keyword content
- CategoryService: category name
- UserService: email address

Manually changed to log only non user controlled tags like Id's

3.13. P-13 — Missing Content-Security-Policy

Severity	Medium
Status	Complete
Source	Peer report, Medium finding 5
Location	No security header configuration; templates contain inline oninput and onsubmit handlers
CWE	CWE-693
OWASP 2025	Reported A06; assignment mapping A02 Security Misconfiguration

Current evidence. Responses lack CSP. Existing inline event handlers prevent an immediately strict script-src 'self' policy.

Risk. If an injection defect is introduced, the browser has fewer restrictions on executable content and external resource loading.

Planned mitigation and rationale. Adopt a restrictive same-origin CSP after moving inline JavaScript to a local static script or using nonces. CSP is defence in depth and does not replace output encoding.

Implementation guide. Inventory resource use; remove inline handlers; configure default-src 'self', restrictive script/style directives, object-src 'none', base-uri 'self', form-action 'self', and frame-ancestors 'none'; begin with report-only if needed.

Verification and report evidence. Content security filter added as part of security filter chain on the server.

3.14. P-14 — Missing anti-clickjacking protection

Severity	Medium
Status	Complete
Source	Peer report, Medium finding 6
Location	No X-Frame-Options or CSP frame-ancestors configuration
CWE	CWE-1021
OWASP 2025	Reported A06; assignment mapping A02 Security Misconfiguration

Current evidence. Application pages can be framed because no response header forbids it.

Risk. A hostile page can visually disguise PromptVault and induce unintended user or administrator actions.

Planned mitigation and rationale. Send both CSP frame-ancestors 'none' and X-Frame-Options: DENY for compatible layered protection.

Implementation guide. Configure headers centrally in the Spring Security filter chain and ensure error responses receive them too.

Verification and report evidence. Adding on from P-13, denying iframes in the header was added on to the SecurityFilterChain.

Then clickjacking.html was used to test the clickjacking which was previously possible.

3.15. P-15 — Session cookie lacks explicit SameSite policy

Severity	Low
Status	Complete
Source	Peer report, Low finding 1
Location	application.properties session-cookie configuration
CWE	CWE-1275
OWASP 2025	A01 Broken Access Control; defence in depth for A08 CSRF

Current evidence. No explicit SameSite, Secure, or deployment-specific cookie settings are present.

Risk. Cross-site cookie behavior may be weaker or vary by browser/ deployment defaults.

Planned mitigation and rationale. Set SameSite=Lax, HttpOnly=true, and Secure=true in the HTTPS production profile. Keep CSRF tokens enabled.

Implementation guide. Use profile-specific properties so local HTTP development remains possible without weakening production. Review cookie path and avoid a broad domain attribute.

Verification and report evidence. Inspect Set-Cookie under production-like HTTPS and confirm logout invalidates the cookie and authenticated requests fail afterward.

3.16. P-16 — Verbose exception and framework error disclosure

Severity	Low
Status	Complete
Source	Peer report, Low finding 2
Location	orElseThrow/runtime exceptions across controllers and services; no custom error handling
CWE	CWE-209 / CWE-755
OWASP 2025	A10 Mishandling of Exceptional Conditions; related A02

Current evidence. Invalid IDs and database length errors can reach the default framework error response.

Risk. Responses can reveal framework, query, path, and internal implementation details and provide unreliable behavior.

Planned mitigation and rationale. Translate expected missing, validation, authorization, conflict, and server errors into generic responses while logging a correlation ID and safe internal details.

Implementation guide. Add centralized exception handling and a generic HTML error page; disable stacktrace/message inclusion in production; never include SQL or attacker input in the client response.

Verification and report evidence. Custom error handler has been implemented exception stacktrace now reduced to this for an invalid access attempt

```
2026-08-31 09:23:03.650 [http-nio-8080-exec-4] ERROR
c.y.p.c.GlobalExceptionHandler ErrorResponse - request failed
correlationId=267f0849-4066-4485-baa2-62c5253f93c9 status=500
exceptionType=org.springframework.web.servlet.resource.NoResourceFoundException
```

3.17. P-17 — Timing-sensitive plaintext password comparison

Severity	Low
Status	Planned
Source	Peer report, Low finding 3
Location	AuthService.login uses String.equals
CWE	CWE-208
OWASP 2025	Not separately listed in OWASP Top 10; related A07

Current evidence. Plaintext strings are compared directly and unknown accounts avoid password verification entirely.

Risk. Response timing may provide an additional authentication oracle, especially when combined with unlimited attempts.

Planned mitigation and rationale. Use Spring Security's password verification and configure a dummy hash path for unknown users so authentication performs comparable expensive work. P-01 and P-09 are prerequisite controls.

Implementation guide. Do not build a custom constant-time password loop. Use the framework provider with BCrypt, generic failures, and throttling.

Verification and report evidence. This is complete as part of the previous vulnerability fixes.

4. Lecturer guideline additions

These are additional applicable gaps that were not identified by the assessor but were present in the OWASP list that the lecturer has put in the submission guidelines.

4.1. T-01 — DB and administrator credentials hard-coded inside application.config

Severity	High
Status	Complete
Source	Lecturer submission guidelines
Location	application.properties
CWE	CWE-798
OWASP 2025	A02 Security misconfiguration and A07 Authentication Failures

Current evidence. server connection string, username and password are committed to the repo.

Risk. Repository access or access to build artefacts can provide attackers with valid admin credentials.

Planned mitigation and rationale. Externalise secrets and use a key-vault provider where applicable

Implementation guide. Use environment placeholder values. Scrub secrets from git history or rotate any credentials. Create separate application.properties for each environment.

Verification and report evidence. In this case, I have opted to have a pretend build script that injects the environment variables in a bash script called pv. In a real environment, secrets should be provided as part of a CI/CD pipeline and robust key rotation system should be in place to reduce chances of a leak.

```
spring.datasource.url=${PV_DB_URL}  
spring.datasource.username=${PV_DB_USERNAME}  
spring.datasource.password=${PV_DB_PASSWORD}
```

Listing 7: Hard coded values were replaced with environment placeholders like such.

5. OWASP Top 10:2025 Coverage Matrix

Category	Applicable findings	Coverage note
A01 Broken Access Control	P-03–P-06, P-15	Ownership-aware data access, deny-by-default routes, active-session revocation, and fixation protection. SSRF is not applicable because the app performs no external URL fetch.
A02 Security Misconfiguration	P-13, P-14, P-16, L-01	Secrets, profiles, headers, safe errors, least privilege, and unnecessary endpoints. No permissive CORS configuration was found.
A03 Supply Chain	P-02	Immediate upgrade plus recurring inventory, scanning, and review.
A04 Cryptographic Failures	P-01	Password hashing, session-data minimization, and HTTPS deployment. Prompt/history storage is functional scope, so retention should be justified rather than falsely described as unnecessary.
A05 Injection	P-10, P-12	Server validation and safe logging. Spring Data repository usage and Thymeleaf <code>th:text/th:value</code> already reduce SQL injection and XSS risk.
A06 Insecure Design	P-09, P-10	Abuse limits, reliable validation, and a reviewable policy workflow.
A07 Authentication Failures	P-01, P-06, P-07, P-09, P-17, L-01	Framework authentication, BCrypt, password policy, throttling, revocation, fixation protection, and expiration.
A08 Data Integrity Failures	P-03–P-05, P-08	CSRF, server-side ownership/state checks, and admin-only policy review. No unsafe deserialization or external update mechanism was found.
A09 Logging Failures	P-11, P-12	Minimal, neutralized, structured, and complete security events.
A10 Exceptional Conditions	P-10, P-16	Typed failures, safe responses, rollback, fail-secure policy handling, and concurrency tests.

6. Existing protections and false positives

- **SQL injection:** persistence uses Spring Data JPA derived queries and fixed JPQL rather than string-concatenated SQL. Parametrised access that does not allow injections.
- **Cross site scripting payloads blocked:** templates currently render untrusted values with escaped `th:text` and `th:value`; no `th:utext` was found. All output follows safe HTML rendering.
- **Path traversal:** user input is not used in filesystem paths.
- **Hardcoded passwords in DataSeeder.java:** Intentional seed data for development. In a real environment this code would only run under dev and test builds. No risk if proper CI/CD practice is taken.
- **Full User entity retained in HTTP session:** CWE-201: Insertion of Sensitive Information Into Sent Data. To mitigate session dump exposing data, the session was modified to only contain minimum required data on a user.

7. Conclusion

The peer assessor provided invaluable feedback on the vulnerabilities of this application. The report covered all 10 OWASP areas and has greatly assisted in securing the Prompt Vault application.